Nama : Cut Alzeena Rency Fadania

NIM : H1D024037

Shift Awal : A
Shift Akhir : F

Tugas Pertemuan 4

1. Knowledge base

```
db_kerusakan = {
    "Gangguan RAM (Memori)": {
        "ciri": ["bunyi_beep", "restart_sendiri", "blue_screen"],
        "penanganan": "Lepaskan RAM, bersihkan bagian pin emas dengan penghapus secara perlahan. Pasang kembali atau pindahkan
    },
    "Masalah Power Supply Unit": {
        "ciri": ["mati_mendadak", "bau_gosong", "kipas_tidak_mutar"],
        "penanganan": "Cek kabel power dan stabilitas listrik. Jika tercium aroma hangus atau kipas mati, segera ganti PSU seb
    },
    "Overheating System": {
        "ciri": ["kinerja_lambat", "mati_saat_berat", "kipas_berisik"],
        "penanganan": "Bersihkan debu pada heatsink dan fan. Pastikan pasta termal (thermal paste) pada prosesor masih layak a
    },
    "Kerusakan Unit Grafis (VGA)": {
        "ciri": ["layar_bergaris", "layar_blank", "game_crash"],
        "penanganan": "Update driver ke versi terbaru. Bersihkan konektor VGA. Jika muncul artefak (garis permanen), biasanya
    },
    "Storage Error (HDD/SSD)": {
        "ciri": ["booting_lama", "file_corrupt", "bunyi_klik"],
        "penanganan": "Segera lakukan backup data ke cloud atau eksternal. Gunakan software pengecek kesehatan disk. Ganti ke
    }
}
```

Dalam sistem pakar ini, Knowledge Base adalah "otak" yang menampung semua dokumentasi teknis yang dibutuhkan sistem untuk berpikir. Secara sederhana, bagian ini menyimpan peta hubungan antara jenis kerusakan hardware, gejala yang muncul di lapangan, hingga solusi *troubleshooting*-nya.

Arsitektur Data

Untuk menyimpan pengetahuan ini, saya menggunakan struktur Dictionary Python dengan format *nested* (bertingkat) agar manajemen datanya lebih rapi:

- Key (Kunci): Sebagai identitas utama, yaitu nama kerusakan hardware (misalnya: "Kerusakan GPU / VGA").
- Value (Nilai): Berisi sub-dictionary yang menyimpan *list* gejala (ciri) dan string narasi untuk solusi perbaikan (penanganan).

Knowledge base menyimpan aturan dalam bentuk hubungan antara gejala dan jenis kerusakan. Setiap kerusakan memiliki beberapa gejala yang menjadi indikator untuk proses diagnosis. Jika sebagian besar gejala yang dimasukkan pengguna sesuai, maka sistem akan mengarah pada kemungkinan kerusakan tersebut. Penggunaan dictionary dipilih karena: -

Strukturnya rapi dan terorganisir -Mudah dipahami -Fleksibel untuk dikembangkan tanpa perlu mengubah logika utama program

2. def analisia_kerusakan

```
def analisa_kerusakan(list_gejala):
    temuan = []

    if not list_gejala:
        return temuan

    for nama_error, info in db_kerusakan.items():
        syarat_error = set(info["ciri"])
        pilihan_user = set(list_gejala)

        # Mencari kecocokan gejala
        match_count = len(syarat_error.intersection(pilihan_user))

        if match_count >= 2:
            temuan.append({
                "kategori": nama_error,
                "tindakan": info["penanganan"]
            })

    return temuan
```

Fungsi analisa_kerusakan ini bekerja sebagai mesin inferensi yang mencocokkan input gejala pengguna dengan basis pengetahuan menggunakan operasi himpunan. Setelah memastikan input tidak kosong, sistem mengubah daftar gejala menjadi tipe data set agar bisa menggunakan fungsi intersection() untuk menemukan irisan gejala secara instan. Diagnosis dianggap valid hanya jika memenuhi ambang batas minimal dua gejala yang cocok (match_count >= 2). Semua kerusakan yang terdeteksi kemudian dikumpulkan dalam sebuah list lengkap dengan solusi penanganannya untuk ditampilkan kembali ke pengguna.

3. GUI

```
self.input_status = {}

# Generate Checkbox secara dinamis
for key, val in indeks_gejala.items():
    status_var = tk.BooleanVar()
    check_btn = tk.Checkbutton(self.box_container, text=val, variable=status_var, bg="#ffffff", font=("Segoe UI", 9), anchor="w")
    check_btn.pack(fill="x", padx=15, pady=3)
    self.input_status[key] = status_var
```

antarmuka secara dinamis dengan melakukan iterasi pada indeks_gejala. Setiap gejala direpresentasikan oleh widget Checkbutton yang status centangnya (True/False) disimpan dalam objek tk.BooleanVar(). Hal ini memungkinkan aplikasi untuk memiliki daftar pertanyaan yang fleksibel dan mudah diperbarui tanpa mengubah kode UI.

4. Pengumpulan Input Pengguna

```
# Ambil list gejala yang dicentang
pilihan = [k for k, v in self.input_status.items() if v.get()]
```

Saat tombol diagnosa diklik, fungsi akan melakukan pemindaian terhadap seluruh checkbox. Sistem menggunakan *list comprehension* untuk mengumpulkan hanya kunci-kunci gejala yang statusnya bernilai True (dicentang). Daftar gejala terpilih inilah yang kemudian dikirim ke mesin inferensi untuk diproses.

5. Inferensi

```
# Mencari kecocokan gejala
match_count = len(syarat_error.intersection(pilihan_user))
```

Logika "berpikir" sistem terletak pada penggunaan operasi irisan (*intersection*) pada tipe data set. Sistem membandingkan kumpulan gejala yang dipilih pengguna dengan gejala yang terdaftar pada database untuk setiap jenis kerusakan. Pendekatan ini memastikan proses pencarian kecocokan berjalan sangat cepat dan efisien secara komputasi.

6. Penentuan Diagnosis

```
if match_count >= 2:
    temuan.append({
        "kategori": nama_error,
        "tindakan": info["penanganan"]
    })
```

menerapkan ambang batas (*threshold*) untuk menentukan validitas hasil. Diagnosis hanya akan dimasukkan ke dalam daftar temuan jika jumlah gejala yang cocok minimal berjumlah dua. Hal ini berfungsi untuk meminimalkan kesalahan diagnosa (false positive) yang mungkin muncul jika gejala yang dimasukkan pengguna terlalu sedikit atau terlalu umum.

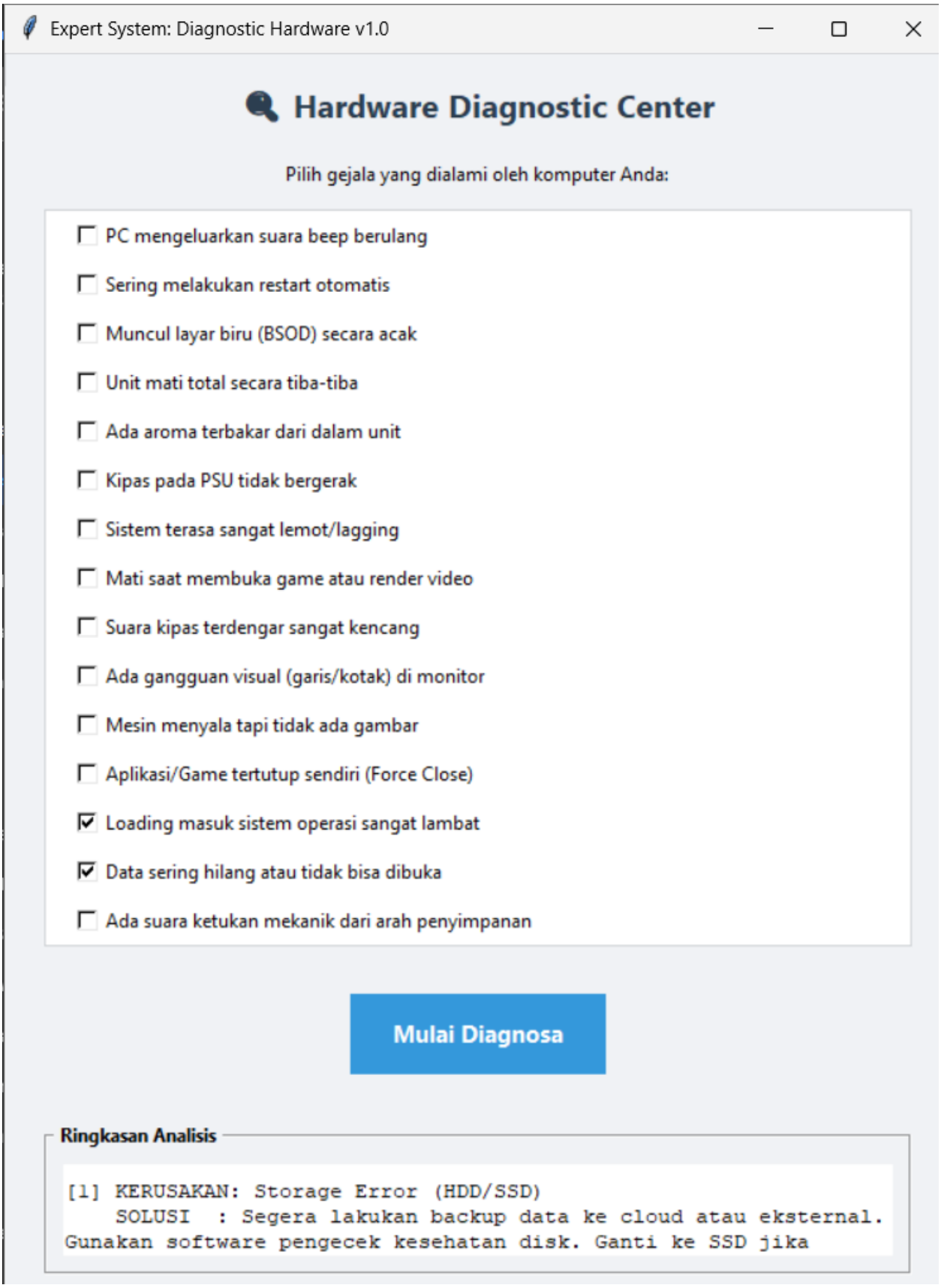
7. Output

```
else:
    self.viewer_teks.insert(tk.END, "LAPORAN DIAGNOSA HARDWARE\n")
    self.viewer_teks.insert(tk.END, "="*40 + "\n\n")

    for i, poin in enumerate(data_hasil):
        self.viewer_teks.insert(tk.END, f"[{i+1}] KERUSAKAN: {poin['kategori']}\n")
        self.viewer_teks.insert(tk.END, f"    SOLUSI  : {poin['tindakan']}\n\n")

self.viewer_teks.config(state="disabled")
```

Setelah proses inferensi selesai, sistem akan memformat data hasil diagnosa ke dalam area teks. Laporan yang ditampilkan mencakup nomor urut, kategori kerusakan yang terdeteksi, serta instruksi tindakan solusi. Jika tidak ada kecocokan yang memenuhi ambang batas, sistem secara otomatis memberikan informasi bahwa pola kerusakan tidak ditemukan.


Expert System: Diagnostic Hardware v1.0
Hardware Diagnostic Center
Pilih gejala yang dialami oleh komputer Anda:
PC mengeluarkan suara beep berulang
Sering melakukan restart otomatis
Muncul layar biru (BSOD) secara acak
Unit mati total secara tiba-tiba
Ada aroma terbakar dari dalam unit
Kipas pada PSU tidak bergerak
Sistem terasa sangat lemot/lagging
Mati saat membuka game atau render video
Suara kipas terdengar sangat kencang
Ada gangguan visual (garis/kotak) di monitor
Mesin menyala tapi tidak ada gambar
Aplikasi/Game tertutup sendiri (Force Close)
Loading masuk sistem operasi sangat lambat
Data sering hilang atau tidak bisa dibuka
Ada suara ketukan mekanik dari arah penyimpanan
Mulai Diagnosa
Ringkasan Analisis
[1] KERUSAKAN: Storage Error (HDD/SSD)
SOLUSI : Segera lakukan backup data ke cloud atau eksternal.
Gunakan software pengecek kesehatan disk. Ganti ke SSD jika